



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Hop-paal: A Formal Frogger Journey with UPPAAL

FORMAL METHODS FOR CONCURRENT
AND REAL-TIME SYSTEMS

Authors: Yana Siao

Advisor: Andrea Manini and Pierluigi San Pietro
Academic Year: 2025-26
Version: 1.0
Release Date: 24.07.2026

Contents

Contents	i
1 Model	1
1.1 General Modeling Choices	1
1.2 Global Declarations	3
1.3 Templates	4
1.4 Channels	5
2 Frogger’s Movement Strategy (Stochastic Version)	6
2.1 Base Policy	6
2.2 Alternative Policy: Safety-Biased Weighting	6
3 Verification Results	7
3.1 Symbolic Version	7
3.2 Stochastic Version — Mandatory Query	8
3.3 Stochastic Version — Comparative Analysis	9
4 UPPAAL Template Figures	10
Bibliography	14

1 | Model

1.1. General Modeling Choices

The game is modeled as a Network of Timed Automata composed of six template classes: `GlobalClock`, `GameManager`, `RowRoad`, `TruckRow`, `DivingTurtle`, and `Frogger`, together with global variables and pure functions encoding parts of the game logic that do not need to be represented as automaton structure. The following design decisions shaped the model as a whole:

Single global clock.

Only one real-valued clock, c , is declared, owned by `GlobalClock`. Every other template reacts exclusively to the `tick!` broadcast and to bounded integer counters derived from it. This was preferred over giving each moving entity its own local clock for two reasons: it keeps the number of clock dimensions constant regardless of how many `RowRoad`/`Truck` instances are composed, keeping the zone graph compact; and, more importantly, it removes any ambiguity in collision detection. If entities evolved on independent local clocks, two clocks could drift within the same time unit, creating a window in which `Frogger` and a hazard share a cell without a shared synchronization point at which the collision is actually observed. Forcing every entity to advance in lockstep on `tick!` guarantees that any collision present in the state space is detected at the tick immediately following it.

Row-confinement by construction.

Objects cannot migrate from one row to another, and this is guaranteed by construction rather than by anything verified at runtime. Each `RowInfo`/`RowState` pair is associated with a fixed row at declaration time: `RowState` stores only the horizontal offsets `x[MAX_PER_ROW]` of the objects belonging to that group, with no `y` field at all, and the row a group belongs to (`rowInfo[id].row`) is a constant, never assigned to. Every function that updates an object's position — `moveRow(id)`, called by `RowRoad(id)`/`Truck(id)` — only ever writes to `rowState[id].x[obj]`; there is no function, in any template, that re-assigns an object from one `rowState[]` entry to another. `Frogger`'s own dragging function, `dragFrogIfRiding`, likewise only updates `frog_x` while `Frogger` rides a floating object, never `frog_y`. Consequently, cross-row movement of a moving object is not a reachable-but-unverified corner case: it is a transition that simply does not exist anywhere in the model's syntax, so no state in the model's state space represents it.

Two extra off-screen cells.

Every row's horizontal domain is $[0, \text{TOTAL_LOOP})$ rather than $[0, \text{WIDTH})$, where `TOTAL_LOOP` =

$WIDTH + BORDER_DELAY$ and $BORDER_DELAY = 2$. This directly encodes the requirement that an object reappearing on the opposite side does so "after a delay equivalent to the time required to traverse two additional cells at its own speed": the two extra cells are folded into the same cyclic coordinate space used on-screen, so an object simply keeps advancing through indices $WIDTH..TOTAL_LOOP - 1$ before wrapping to 0. Frogger's own coordinate stays in $[0, WIDTH)$, but it can be dragged through the transit cells while riding a floating object, reproducing the required off-screen carry of difficulty B without extra locations or clocks.

Movement and diving as separate templates.

Vehicles, logs, turtles, and diving turtles all share the same displacement rule (advance one cell every speed time units, in a fixed direction, with wraparound), captured uniformly by `RowRoad`. The diving cycle of diving turtles is a logically independent concern — it does not affect horizontal position, only whether the object is safe to stand on — so it is implemented in a separate template, `DivingTurtle`. This keeps `RowRoad` a single reusable template for every moving row and avoids duplicating diving logic across the two diving-turtle groups (rows 7 and 10).

State abstraction on `DivingTurtle` and `Frogger`.

Both templates originally had one location per phase described in the specification (four diving phases; several Frogger sub-states for waiting, choosing a move, and evaluating collisions). This exploded the state space, with several queries failing to terminate within a reasonable time (beyond 10 minutes). Both were collapsed to the minimum number of observationally distinct states relevant to collision detection:

- **`DivingTurtle`** distinguishes only *AboveWater* (emerged + diving + emerging, $DIVE_THRESHOLD_ABOVE_WATER = 20$ time units) from *Submerged* ($DIVE_THRESHOLD_SUBMERGED = 10$ time units), since `checkCollision()` only ever needs to know whether a diving turtle is currently safe or deadly, never which of the four original phases is active.
- **`Frogger`** distinguishes only *Respawning* from *Playing* (plus the zero-time committed location *EvaluateSafety*, used to evaluate `checkCollision()` immediately after a move), since every death, timeout, or bay-entry outcome ultimately routes back to the same respawn-and-wait behavior.

This preserves every property needed for verification while removing state that only tracked intermediate phases with no observable effect on collisions, gameplay, or scoring.

Turtle groups instead of individual turtles.

Rows 7 and 10 each contain two turtles (one of which dives) separated by a one-cell gap. Rather than modeling each turtle independently, each pair-plus-gap is modeled as a single floating-object group of size 5 (row 7) or size 3 (row 10). The cell between two turtles of the same group carries no distinguishing property: Frogger can stand there exactly as on either turtle, so the model only needs to know whether Frogger's cell falls within the group's occupied span, not which specific turtle is underneath. This is behaviorally equivalent to modeling turtles individually for every property of interest, while using one

`RowInfo/RowState` entry (and one dragging check) instead of two or three.

GameManager decoupled from per-tick logic.

`GameManager` contains no collision or movement rules, and reacts to `tick?` only for 2 seconds buffer time waiting. It is triggered by the discrete events broadcast by other templates (`reachBay?`, `loseOneLife?`, `timeOut?`) and is responsible exclusively for book-keeping: score, lives, level progress, and the mandatory respawn delay. This keeps all meta-game state out of the templates that reason about position and safety.

Truck as a distinct template.

Trucks occupy a single row of the map and are subject to the same collision rules as any other vehicle, so their positional and wraparound logic is identical to `RowRoad`'s. However, the stochastic version requires the time between two consecutive truck moves to be exponentially distributed with rate $\lambda = 1/\text{speed}$, which cannot be expressed through `globalTick % speed == 0` guard. Truck therefore reuses `RowRoad`'s movement/wraparound/dragging logic but replaces the periodic guard with an exponential-rate location, so that the two versions of the model share an identical row layout and differ only in how the truck row's movement is triggered.

Compliance with generalization and documentation requirements.

All map dimensions, timing constants, scoring constants, and per-row parameters (size, speed, direction, type, initial positions) are declared once as named constants or entries of the `rowInfo[]` table in the global declarations, and are never hardcoded inside a template; changing a single constant (e.g. `WIDTH`, `RESPAWN_DELAY`, or a row's speed) propagates consistently to every template instance. Every function in the declarations is documented with a Javadoc-style comment describing its parameters, return value, and behavior, and every template contains at least one comment explaining its role in the model.

1.2. Global Declarations

The map is represented implicitly through two constant arrays indexed by row: `map[HEIGHT]` gives each row's terrain type (`SAFE`, `ROAD`, `RIVER`, `HOME`), and `rowFirstGroup[HEIGHT]` gives the index into `rowInfo[]` of the first moving-object group in that row (or -1 for rows with no moving objects). Each moving-object group is described by an immutable `RowInfo` record (`row`, `size`, `speed`, `direction`, `type`, diving flag, number of objects, initial positions) and a mutable `RowState` record holding only the objects' current horizontal positions; keeping immutable configuration and mutable runtime state in separate structures avoids re-deriving fixed parameters at every step and keeps the state vector limited to what actually changes. Collision detection (`checkCollision`, `getObjectAtPosition`, `isCellOccupied`) and bay bookkeeping (`getHomeBayIndex`, `occupyHomeBay`, `allBaysOccupied`) are implemented once in the global declarations rather than duplicated per template, since both Frogger and the "without considering Frogger" queries need to evaluate the same rules against the shared `rowState`.

1.3. Templates

All template figures are reported in the "UPPAAL Template Figures" section.

- **GlobalClock.** Single location, *Running*, with invariant $c \leq \text{TICK_PERIOD}$. On $c == \text{TICK_PERIOD} \wedge \text{globalTick} < \text{currentLevelTimeLimit}$, it broadcasts `tick!`, resets c , and increments `globalTick`. On $\text{globalTick} \geq \text{currentLevelTimeLimit}$, it broadcasts `timeOut!` and resets both c and `globalTick`, starting the timer for the next attempt.
- **GameManager.** Three locations: *Start*, *Playing* (invariant $\text{lives} > 0$), and *WaitingRespawn*. From *Start*, `initializeGame()` resets `lives`, `score`, and `level`, then control moves to *WaitingRespawn*. From *Playing*: `reachBay?` awards `BAY_SCORE` and, if all bays are now occupied, additionally awards `LEVEL_SCORE` and calls `finishCurrentLevel()`; `loseOneLife?` and `timeOut?` both call `loseLife()`. All three transitions lead to *WaitingRespawn*, where a local counter `frogMoveTimer` increments on `tick?` until it reaches `RESPAWN_DELAY`, at which point `frogInitPosition!` is broadcast and control returns to *Playing*. *WaitingRespawn* is the only non-committed location, since it is the only one where real time must actually elapse.
- **RowRoad.** Parametric template `RowRoad(const int id)`, instantiated once per moving row except trucks. Single location *Running*; on `tick?`, `updateRow(id)` calls `moveRow(id)` only if $\text{globalTick} \% \text{rowInfo}[id].\text{speed} == 0$. `moveRow` first drags Frogger by one cell if it is riding the row ($\text{log}/\text{turtle}/\text{diving-turtle row and frog_y} == y$), then shifts every object in the row by one cell, wrapping modulo `TOTAL_LOOP`. Objects are never reset once the game starts, so the template has no other location.
- **TruckRow.** Same positional/wraparound/dragging logic as *RowRoad*, applied to the single truck row, but its *Waiting* location carries an exponential rate $1 : \text{TRUCK_SPEED}$ instead of a tick-counted guard, so the delay until `moveRow(id)` fires is drawn from an exponential distribution with mean `TRUCK_SPEED`.
- **DivingTurtle.** Single shared process (one instance controls both diving-turtle groups). Two locations, *AboveWater* and *Submerged*, with local counter `timer`. `timer` increments on `tick?` in the current state; when it reaches `DIVE_THRESHOLD_ABOVE` (in *AboveWater*) or `DIVE_THRESHOLD_SUBMERGED` (in *Submerged*), the automaton switches state, resets `timer`, and updates the global `isSubmerged` flag accordingly.
- **Frogger.** Three locations: *Respawning*, *Playing* (invariant $\text{lives} > 0$), and *EvaluateSafety* (committed). In *Respawning*, Frogger waits for `frogInitPosition?`, then calls `frogReposition()` and resets `globalTick`. In *Playing*, a local counter `timerBetweenMoves` enforces Assumption 2 (at least 1 time unit between consecutive moves); once it reaches `TICKS_BTW_MOVES` and the level timer has not expired and the current cell is not already a collision, `moveFrog()` is invoked and control passes to *EvaluateSafety*. Two additional self-transitions from *Playing* handle the level timer expiring and an immobile Frogger being caught by an approaching hazard (`loseOneLife!`). In *EvaluateSafety*, `checkCollision()` is evaluated with zero elapsed time against the post-move state: an unsafe cell broadcasts `loseOneLife!`;

a free home bay broadcasts `reachBay!` and calls `occupyHomeBay()`; otherwise the move is a plain safe move, `score += MOVE_SCORE` awarding only upward movements, and control returns to *Playing*.

The symbolic and stochastic models share the same environment templates but intentionally use different Frogger controllers: deterministic upward movement for exhaustive verification and probabilistic four-direction movement for SMC.

1.4. Channels

All channels are broadcast channels.

Channel	Emitter	Observers	Purpose
<code>tick</code>	<code>GlobalClock</code>	<code>RowRoad</code> , <code>Truck</code> , <code>DivingTurtle</code> , <code>Frogger</code> , <code>GameManager</code>	Advances every process by one discrete time step
<code>timeOut</code>	<code>GlobalClock</code>	<code>Frogger</code> , <code>GameManager</code>	Signals level-timer expiry
<code>frogInitPosition</code>	<code>GameManager</code>	<code>Frogger</code>	Ends the respawn delay, resumes play
<code>reachBay</code>	<code>Frogger</code>	<code>GameManager</code>	Reports a successful bay entry
<code>loseOneLife</code>	<code>Frogger</code>	<code>GameManager</code>	Reports a fatal collision or timeout

Table 1.1: Channel declarations and synchronization roles.

2 | Frogger's Movement Strategy (Stochastic Version)

2.1. Base Policy

The base stochastic model implements the movement policy fixed by the assignment: at every eligible tick, Frogger's branchpoint chooses UP with probability 0.5, LEFT with 0.2, RIGHT with 0.2, and DOWN with 0.1, independently of the current board state. This policy is used to answer the mandatory stochastic query and as the baseline against which an alternative policy is compared.

2.2. Alternative Policy: Safety-Biased Weighting

The improved model (`FM_v8_stochastic_improved.xml`) replaces the four fixed probability labels with weight functions evaluated at each branch: `weightUp()`, `weightLeft()`, `weightRight()`, `weightDown()`. Each direction is assigned a base weight preserving the original policy's relative priority (`BASE_UP = 5`, `BASE_LEFT = 2`, `BASE_RIGHT = 2`, `BASE_DOWN = 1`, matching the 0.5 : 0.2 : 0.2 : 0.1 ratio). On top of the base weight, a safety bonus is added whenever the corresponding move is currently safe — i.e. it does not lead to an immediate collision, computed by mirroring `checkCollision()`'s per-tile logic against the hypothetical destination cell without mutating Frogger's state (`safeUp`, `safeDown`, `safeLeft`, `safeRight`). The bonuses are `BONUS_UP = 12`, `BONUS_SIDE = 5`, `BONUS_DOWN = 2`: UP receives the largest bonus so that Frogger still preferentially makes forward progress whenever doing so is not risky, while unsafe directions fall back to a minimal weight of 1 (rather than 0) so that no branch is ever completely excluded. This keeps the branchpoint's structure and the four possible actions identical to the base policy — only the probability weights differ, and only as a function of currently-observable safety, not of any lookahead.

3 | Verification Results

3.1. Symbolic Version

The symbolic model is checked with exhaustive model checking rather than SMC, using UPPAAL’s standard reachability/safety operators ($A[]$ and $E<>$) instead of probabilistic ones. Because exhaustive verification requires exploring the full state space rather than sampling it, Frogger’s movement in this version is intentionally simplified to a single deterministic action, `moveFrog()`, which only attempts to move up (`canMoveUp()`; no left, right, or down transitions, and no probabilistic branch point). This is sufficient for checking reachability and safety properties, since horizontal displacement of Frogger is still fully exercised through `dragFrogIfRiding()` whenever it rides a log or turtle group, and keeping movement deterministic avoids inflating the state space with a branching factor that plays no role in the properties being verified. The full four-direction, weighted movement policy is introduced only in the stochastic version (Section 3.2), where it is required to produce a statistically meaningful gameplay simulation.

Two of the seven properties (I and II) concern only the environment—deadlock-freedom and confinement of moving objects to their own row—and are checked with Frogger removed from the system declaration; adding Frogger would only inflate the state space explored without affecting the truth of these two properties.

Table 3.1: Symbolic Verification Properties and Results

#	Formula	Property checked	Verification time
I	$A[]$ not deadlock	Without Frogger, the environment (moving rows, diving turtles, game manager) never reaches a deadlock state.	20 s (80 s on a less powerful machine)
II	$A[]$ forall(y) forall(obj) rowState[y].x[obj] >= 0 && rowState[y].x[obj] < TOTAL_LOOP	Without Frogger, every moving object’s horizontal position always stays within its row’s cyclic bounds.	10 s (40 s)
III	$E<>$ bayOccupied[0]	Frogger can eventually reach the first home bay, at cells (0,12)–(1,12).	8 s (37 s)

Table 3.1: Symbolic Verification Properties and Results (continued)

#	Formula	Property checked	Verification time
IV	$E\langle \rangle ((\text{frog_y}==12 \ \&\& \ (\text{frog_x}==8 \ \ \text{frog_x}==9)) \ \&\& \ \text{score}==17)$	Frogger can score 17 points reaching the home bay in position (8,12)–(9,12).	50 s (200 s)
V	$E\langle \rangle (\text{lives}==3 \ \&\& \ \text{frog_y}==6)$	Frogger can reach a safe cell in row 6 after already having lost two lives.	10 s (45 s)
VI	$E\langle \rangle (\text{globalTick} \geq 10 \ \&\& \ \text{lives}==5 \ \&\& \ \text{getObjectAtPosition}(\text{frog_y}, \text{frog_x}) \neq -1 \ \&\& \ \text{rowInfo}[\dots].\text{type}==\text{LOG})$	Frogger may be standing on a log, with all five lives still intact, 10 seconds after the game starts.	1 s (10 s)
VII	$E\langle \rangle (\text{Frog.EvaluateSafety} \ \&\& \ \text{checkCollision}() \ \&\& \ \text{isSubmerged} \ \&\& \ \text{rowInfo}[\dots].\text{diving})$	Frogger can die by riding a diving turtle at the exact instant it becomes submerged—the only instant this is observable in <code>EvaluateSafety</code> before the automaton returns Frogger to Respawn.	1 s (1 s)

3.2. Stochastic Version — Mandatory Query

Query I: $\Pr[\leq 1000](\diamond(\text{level} > 1 \wedge \text{score} \geq 200))$: probability of completing the first level while scoring at least 200 points within 1000 time units.

- **Base policy (FM_v7):** ≤ 0.0499441 (95% CI)
- **Safety-biased policy (FM_v8):** ≤ 0.0499441 (95% CI)

Both policies yield the same low upper bound: reaching 200 points requires roughly 40 successful up-moves or several completed bay/level cycles within a strict 1000-time-unit horizon (30-second level timer at 0.1 s/unit), which neither policy achieves with meaningful probability given the respawn delay and death rate. This indicates the bound is dominated by the time budget rather than by the specific movement policy.

3.3. Stochastic Version — Comparative Analysis

Two aspects were analyzed to compare the base and safety-biased policies over the same 1000-time-unit horizon: (a) how deep into the river Frogger typically progresses, and (b) the maximum score achieved per run.

(a) Reachability of each row ($\Pr[\leq 1000](\diamond \text{frog_y} == k)$) for $k = 6..12$

The safety-biased policy reaches every river row with substantially higher probability than the base policy, and the gap widens with row depth (e.g. row 9: 0.149 vs 0.839; row 12: ≤ 0.05 vs 0.241). This confirms that biasing Frogger’s random walk toward locally safe moves meaningfully increases progress through the river section, where a wrong move is immediately fatal, without changing the underlying probability weights’ relative ranking.

Row (k)	Base Policy	Safety-Biased Policy
6 (safe, pre-river)	0.751 ± 0.048	≥ 0.950
7 (turtles)	0.798 ± 0.047	≥ 0.950
8 (logs)	0.338 ± 0.049	0.949 ± 0.038
9 (long log)	0.149 ± 0.046	0.839 ± 0.046
10 (turtles)	0.046 ± 0.037	0.554 ± 0.050
11 (logs)	0.036 ± 0.034	0.386 ± 0.049
12 (home row)	≤ 0.0499	0.241 ± 0.048

Table 3.2: Row reachability comparison probabilities.

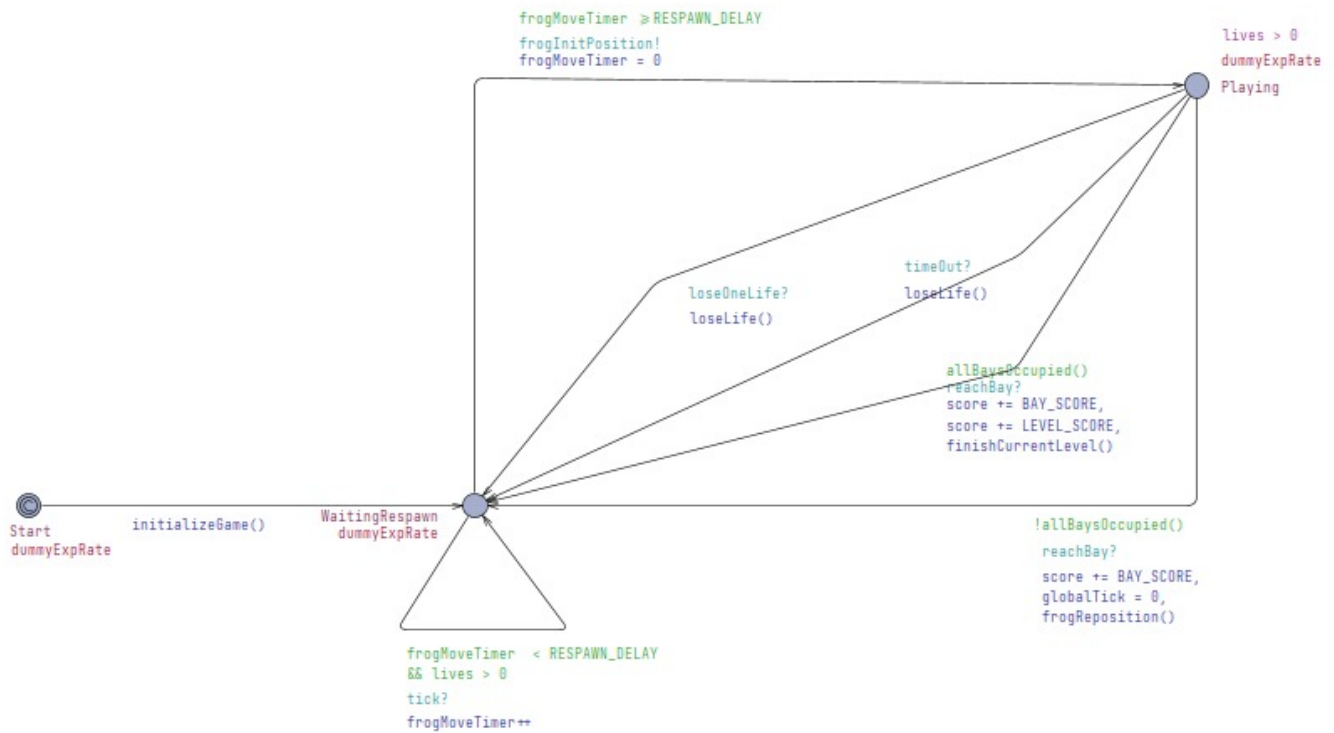
(b) Maximum score per run ($E[\leq 1000; 100](\text{max : score})$)

- **Base policy:** 19.9 ± 1.30
- **Safety-biased policy:** 47.8 ± 2.89

The safety-biased policy roughly doubles the average maximum score, consistent with it surviving longer and reaching deeper rows more often (each successful upward move contributes `MOVE_SCORE`, and reaching a bay contributes `BAY_SCORE`). Both models also confirm $E[\leq 1000; 100](\text{max : level}) \approx 1$ (no run completes level 2 within the horizon under either policy) and $\Pr[\leq 1000](\diamond (\text{lives} = 0)) \geq 0.950056$ (both policies exhaust all five lives with high probability within 1000 time units), showing that the safety bias improves progress and score without being strong enough to prevent eventual failure over a long enough horizon. Per-bay occupancy ($\Pr[\leq 1000](\diamond \text{bayOccupied}[i])$, $i = 0..4$) is consistently near the ≤ 0.05 lower bound under the base policy, while under the safety-biased policy it rises to the 3–9

4 | UPPAAL Template Figures

The following figures provide enlarged views of the UPPAAL templates used in the model.



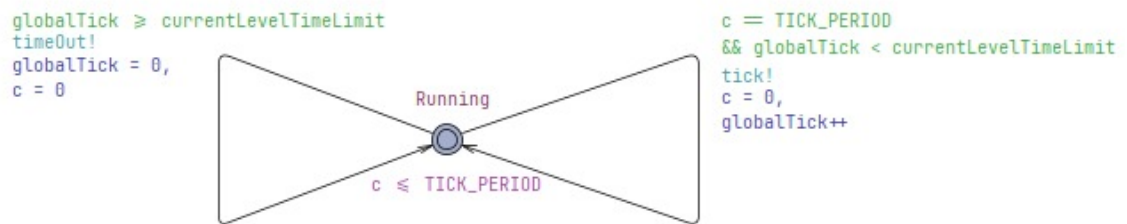


Figure 4.2: GlobalClock template

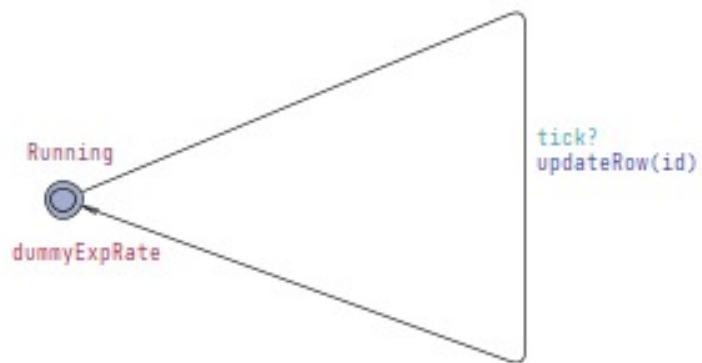


Figure 4.3: RowRoad template

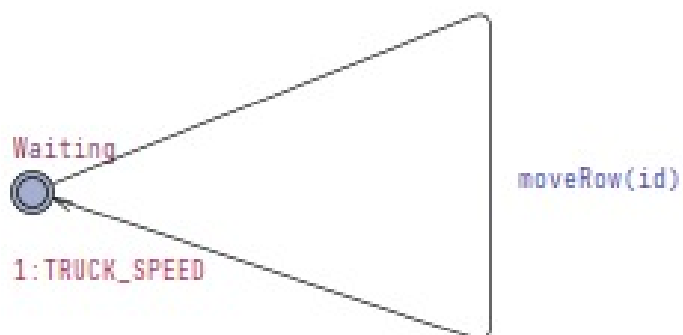


Figure 4.4: TruckRow template

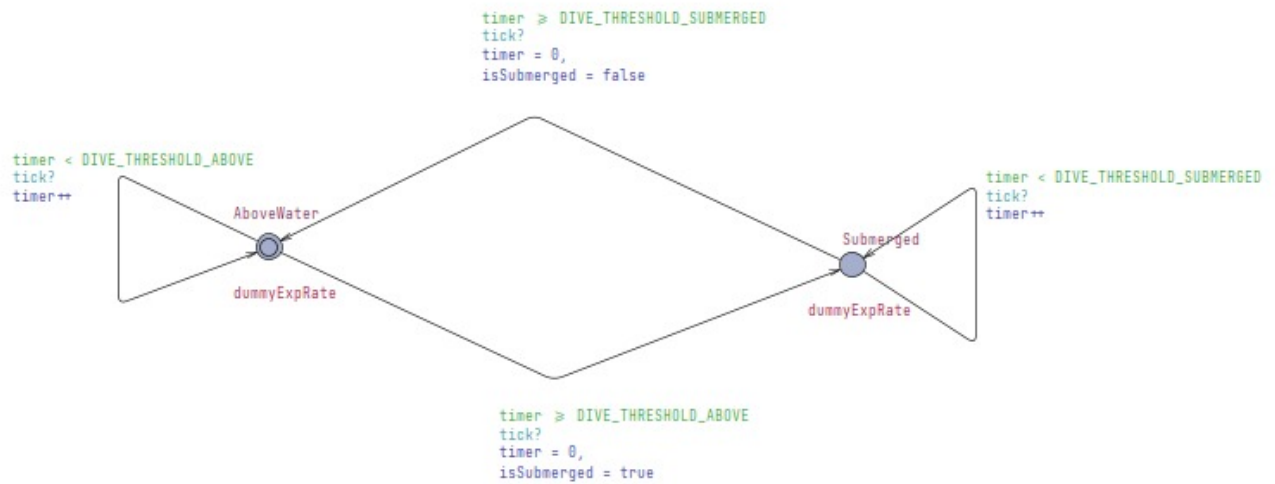


Figure 4.5: DivingTurtle template

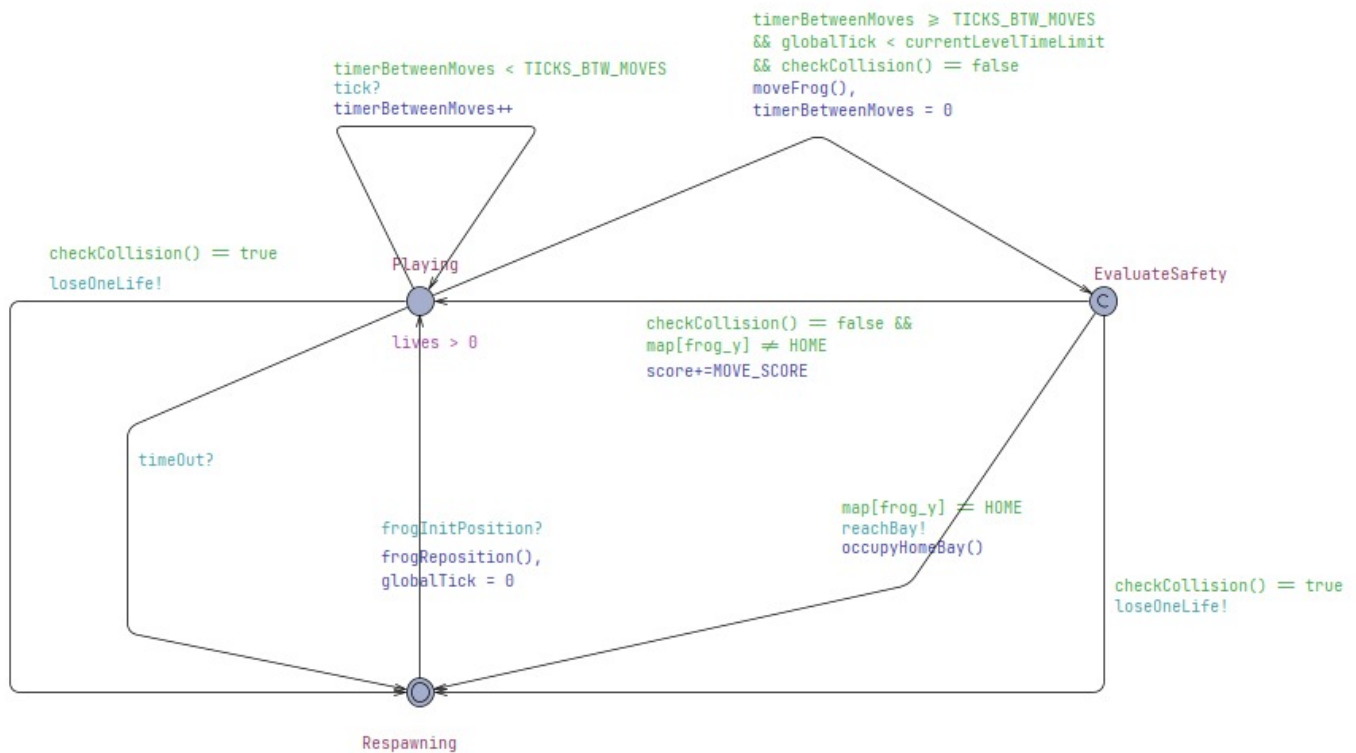


Figure 4.6: Symbolic Frogger template

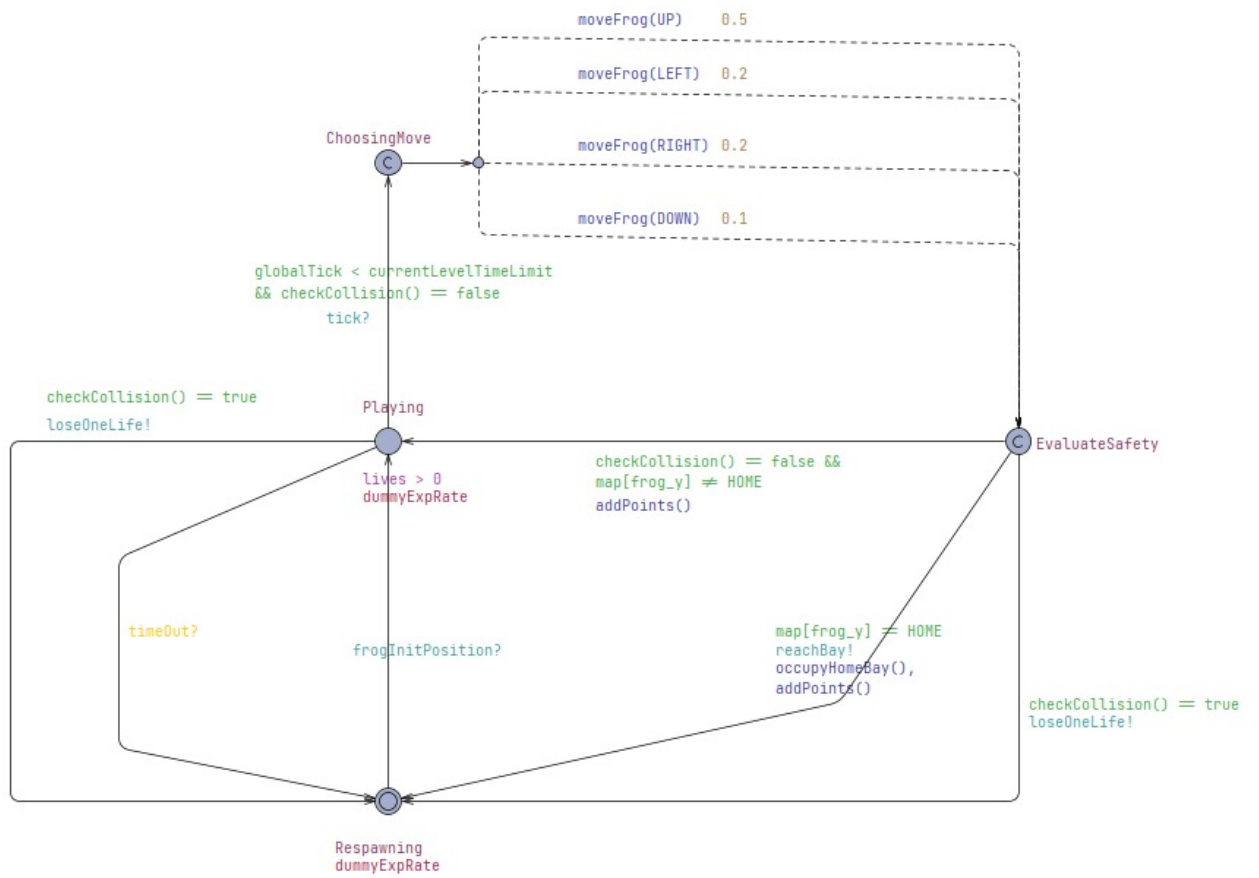


Figure 4.7: Stochastic Frogger template

Bibliography

- [1] A. David, K. G. Larsen, A. Legay, M. Mikučionis, and D. B. Poulsen, *UPPAAL SMC Tutorial*. International Journal on Software Tools for Technology Transfer, 2015.t: <https://uppaal.org/texts/uppaal-smc-tutorial.pdf>
- [2] G. Behrmann, A. David, and K. G. Larsen, *A Tutorial on UPPAAL*.: <https://uppaal.org/texts/new-tutorial.pdf>
- [3] UPPAAL Tutorial Video: <https://youtu.be/0ioBpqDG0f0>
- [4] UPPAAL Modeling Tutorial Video: <https://youtu.be/Kt981QU47zw>